Rheinisch-Westfälische Technische Hochschule Aachen
Alessandro Berti

# Phase Review: 2nd Sprint of the Implementation of "Conformance Predictions"

Group 4:
*Richard Schuppener, Cem Colak*

# Contents

**1 Summary of Sprint 2** **3**

**2 Codebase Structure** **4**

**3 Testing** **5**

**4 Requirements Coverage Status - Overview per Module** **5**
4.1 IO . . . . . 5
4.2 Feature Extraction . . . . . 6
4.3 Model Training . . . . . 8
4.4 Evaluation . . . . . 8
4.5 Config & Pipeline . . . . . 9
4.6 Project Infrastructure & Testing . . . . . 10

**5 User Guide** **11**

**6 Team settings and Workflow** **11**
6.1 Changes in Responsibilities . . . . . 11
6.2 Workflow Improvements for the next sprint . . . . . 12

**7 Exemplary Visualizations and Outputs** **12**
7.1 Visualizations . . . . . 12
7.2 CLI Output . . . . . 14

**8 Sprint review** **17**
8.1 Richard Schuppener . . . . . 17
8.2 Cem Colak . . . . . 17

# 1 Summary of Sprint 2

In this sprint, we focused on implementing all the remaining functionalities required for a fully functional end-to-end pipeline for conformance predictions. We intend to use the final sprint to further refine and optimise these functionalities, so that we can deliver a well-aligned and polished software artifact that fulfils Alessandro's requirements in the most sophisticated way.

Key achievements of this sprint include:

1. Full end-to-end support for all three prediction targets – *Future Conformance Deviations*, *Process Outcome*, and *Remaining Time* – with the CLI, pipeline, feature extraction, and model training extended accordingly.

2. Introduction of a prefix-generation module (`feature_extraction.generate_prefixes`) supporting both *exactly-k* and *varying* $1..k$ prefix modes, with outcome-leakage gating that prevents the target activity from ever appearing inside the feature vector.

3. Implementation of the *Case Attributes* and *Hybrid* feature paths, including robust detection of trace-level attributes for both prefixed (`case:`) and unprefixed (XES-style) columns.

4. A configurable deviation cost threshold $T$ for the Future Deviations target, backed by a corrected cost function that excludes silent ($\tau$) routing moves from the real deviation count.

5. Per-prefix *remaining-time* regression labels and dedicated temporal predictors (elapsed time, time since last event, event count).

6. Extension of the ML model menu beyond the Decision Tree: *Logistic Regression* and *Gradient Boosting* for classification, and *Random Forest*, *Gradient Boosting*, and *Decision Tree Regressor* for regression.

7. Runtime and memory instrumentation via `time.perf_counter` and `psutil`, with optional persistence of the collected measurements as a structured JSON report or as a pie-chart.

8. Prefix-length performance analysis (`metrics_by_prefix_length`) that reports the full metric set per prefix length for a single pooled model.

9. Pytest suite covering feature extraction (case attributes, prefix gating, remaining time, deviation cost), model training (parametrised over all classifier and regressor variants), evaluation metrics, and IO.

10. Extraction of a dedicated *Visualization* module that bundles process-model, decision-tree, confusion-matrix, per-class accuracy rendering etc.

# 2 Codebase Structure

We enriched the initial codebase structure outlined in the first sprint review with a dedicated visualization module that bundles all visualization functionalities together. The modules are decomposed into the following categories:

1. **IO:** Handles inputs and outputs, such as reading event logs and normative models, as well as visualizing process models.

2. **Feature Extraction:** Performs conformance checking (alignments only, hence no separate conformance module), extracts features related to alignments, control flow, and case attributes, and preprocesses the data for ML training (e.g. scaling of numerical values).

3. **Model Training:** Trains a model (as specified by the user) on the target and feature vector produced by the Feature Extraction step. This module also provides the capability to predict unseen data (test data or new data).

4. **Evaluation:** Computes evaluation metrics such as accuracy, precision, recall/fitness, confusion matrix, and ROC.

5. **Config:** Collection of Enums that correspond to the options presented to the user via the CLI. It keeps the codebase clean and easier to manage.

6. **Pipeline:** Bundles functionalities from the other modules into the larger "steps" of the process. Based on the user's choices, the correct functions are called and their inputs and outputs are streamlined together.

7. **Main / CLI:** The entry point of the program; also serves as the guide and option selector for the user. The CLI presents options, numerical metrics, and comprehensive error messages.

8. **Visualization:** Bundles all visualization functionalities, allowing us to maintain and adjust the visualizations in a more modular manner.

# 3 Testing

We discussed with Alessandro at the weekly meetup that we wanted to set up a GitLab runner that runs independently of Richards PC uptime. Thankfully, registering a new runner and creating a token that links the runner to the repository is very straight-forward with GitLab. Alessandro agreed to host such a runner on an RWTH-server, such that we do not face the issue of pending pipelines anymore. This set-up went smoothly, and now our testing pipeline, which runs all the tests in the `tests/` folder with a fresh `python:3.14.4` image and pytest is up and running. The CI pipeline works in the following manner:

1. Navigation into the `Code/` folder: `cd Code`
2. Creation of a temporary virtual environment: `python3 -m venv .ci_venv`
3. Activating this virtual environment: `source .ci_venv/bin/activate`
4. Installation of all required dependencies:
   (a) `pip install --upgrade pip`
   (b) `pip install -r requirements.txt`
   (c) `pip install pytest`
5. Running the tests: `pytest tests/`

We aim to cover as much code as possible with our automated testing pipeline, though a 100% coverage is 1. infeasible and 2. much of our functionality is based on highly tested dependencies. Furthermore, we test by running the CLI manually and watching out for any unexpected errors. After all, we do not expect users to deliberately break the application, but use it in its designated fashion. Thus testing different input data and selected option combinations manually is definitely "good enough" for a student project.

# 4 Requirements Coverage Status - Overview per Module

## 4.1 IO

**All Covered**

- **LL-01.1**: `io.discover_process_model()` supports Inductive, Alpha, and Genetic Miner via PM4Py, enabling automatic model discovery when no normative model is available.
- **LL-01.2**: `visualization.visualize_process_model()` renders the model as Petri net, BPMN, or Process Tree, with optional file saving or direct display.
- **HL-05**: Event log loading, model import, and model discovery are all functional; the foundational data-acquisition step of the pipeline is complete.
- **LL-05.1**: `pm4py.read_xes` for XES logs; `pd.read_csv` combined with `pm4py.format_dataframe` for CSV logs in `io.load_event_log()`.
- **LL-05.2** (partially): Petri net loading via `pm4py.read_pnml` implemented; BPMN files loaded via `pm4py.read_bpmn` and Process Tree files via `pm4py.read_ptml`, both internally converted to a Petri net.
- **LL-05.3**: Inductive, Alpha, and Genetic Miner available as fallback discovery algorithms when no normative model file is provided.
- **HL-04**: `try/except` blocks in `main.py` catch and report setup errors, preprocessing failures, and pipeline exceptions to `stderr` with descriptive labels.
- **HL-01**: End-to-end prediction of conformance deviations is now fully operational; the *Future Deviations* target is wired through the complete pipeline – label extraction via `real_deviation_cost`, configurable threshold $T$, classification training, and evaluation.
- **LL-05.2**: Standalone loading of BPMN and Process Tree files as direct model inputs (without forced internal conversion to a Petri net).

## 4.2 Feature Extraction

### Covered

- **HL-06** (partially): `feature_extraction.compute_alignments()` invokes `pm4py.conformance_diagnostics_alignments`; log-moves, model-moves, and synchronous moves are extracted per case. Distance-to-completion indicators are not yet included.

- **LL-06.1**: PM4Py alignment dictionaries are computed for every trace against the normative Petri net using the standard cost function. Alignments are now keyed by case ID and re-emitted in first-appearance order, eliminating the silent ordering issue that previously misaligned diagnostics with cases.
- **LL-06.2**: Alignment moves are parsed by `real_deviation_cost()` to count genuine deviations (log-moves and visible model-moves) per case; cumulative alignment cost is extracted while silent ($\tau$) routing moves are correctly excluded.
- **HL-07**: `feature_extraction.extract_features()` now supports control-flow features (Bag-of-Activities), alignment-based diagnostic features (Bag-of-move-types per activity), case-attribute features, and the hybrid combination of all three.
- **LL-07.3**: Control-flow frequency matrix constructed via `pd.crosstab` over case IDs and activity names, producing a per-case Bag-of-Activities feature matrix.
- **HL-13**: Alignment results are computed once per run and stored in memory; they are passed as a variable to avoid redundant re-computation within a single pipeline execution.
- **HL-02**: Structured case attributes are fully integrated as predictive features. `list_case_attributes` and `summarize_case_attributes` detect both explicitly prefixed (`case:`) and constant-within-case attributes (XES-style), and the CLI lets the user select a subset of them.
- **LL-07.1**: `generate_prefixes()` expands each case into varying-length prefix pseudo-cases, with optional outcome-leakage gating, supporting both *exactly-k* and *varying* 1..$k$ modes.
- **LL-07.2**: `extract_remaining_time()` and `extract_temporal_features()` together produce a numerical feature matrix tailored to the remaining-time regression target, including elapsed time, time since the previous event, and event count.
- **HL-06**: *We left this out.* Addition of distance-to-completion indicators (e.g. remaining enabled transitions, token distance to the final marking).

**Planned in Sprint 3**

- **LL-13.1**: *Optional:*Hash-based caching of prefix activity sequences to deterministically identify and skip redundant alignment computations.
- **LL-13.2**: *Optional:* Disk-based persistence of alignment results via `joblib`.

## 4.3 Model Training

**Covered**

- **HL-08**: `train_classification_model()` wraps a configurable classifier (Decision Tree, Logistic Regression, or Gradient Boosting) in a `sklearn.Pipeline`. The binary deviation label is derived from the case's real alignment cost via the configurable threshold $T$, so the target is fully driven by alignment-based deviation cost.
- **LL-08.1**: `StandardScaler` is integrated as the first step in every sklearn Pipeline, ensuring scale-consistent feature distributions.
- **HL-09**: Baseline (control-flow features only) and conformance-aware (diagnostics or hybrid features) model variants are both supported via the `PredictionFeature` configuration.
- **LL-08.2**: Additional ML algorithms are available beyond the Decision Tree: *Logistic Regression* and *Gradient Boosting* for classification, and *Random Forest*, *Gradient Boosting*, and *Decision Tree Regressor* for regression. All are exposed via the CLI and selectable per run.

## 4.4 Evaluation

**Covered**

- **HL-16**: `evaluation.compute_classification_metrics()` computes accuracy, precision, recall, F1-score, ROC-AUC, and Brier score for binary and multi-class classification targets.
- **HL-12**: Confusion matrix heatmap and per-class accuracy bar chart are generated via Matplotlib in `visualization.save_evaluation_visuals()`.
- **HL-14**: The system allows for precise measuring of runtime and memory usage.

- **HL-14.1**: Each critical and potentially time-consuming step (often the pipelines) is precisely measured and the runtime is appended to running list. This functionality is performed by `measure_runtime`, which utilizes `time.perf_coutner` internally.
- **HL-14.2**: The peak memory consumed by the program is tracked by the function `measure_memory`. Additionally, it accounts for OS dependent units (bytes or kilobytes).
- **HL-14.3**: A counter accumulates the runtime from different steps in out program: from heavy *PM4Py* methods for discovery, test-train split etc., to whole pipelines that extract features or train models.
- **HL-14.4**: The generated performance data (runtime, peak memory usage) is printed to the CLI and can optionally be saved to a JSON. The runtime can also be visualized as a pie-chart.
- **HL-15**: The main pipeline uses `pm4py.split_train_test` with a user-configurable train ratio (default 0.7).
- **LL-16.2**: `compute_mae`, `compute_mse`, and `compute_rmse` are available in the evaluation module for evaluating remaining-time regression targets.
- **HL-17**: `evaluation.metrics_by_prefix_length()` slices the predictions of a single pooled model by prefix length and reports the full metric set per length; the CLI prints this table whenever prefix lengths vary.

**Planned in Sprint 3**

- **HL-16**: Conformance quality metrics for the normative model (fitness), and wiring of the regression metrics into the pipeline's `compute_regression_metrics` hook so they are reported automatically for the remaining-time target.

## 4.5 Config & Pipeline

**Covered**

- **HL-03**: `main.py` provides an interactive CLI that guides the user through model path/discovery selection, prediction target (future deviations, process outcome, remaining time), feature basis (including case attributes), prefix mode, train/test split ratio, and ML algorithm configuration.

- **LL-03.1**: Prediction target and feature basis options are fully defined in `config.py` and selectable in the CLI; both the case-attribute feature path and the conformance-deviation target are now fully wired through the pipeline.
- **HL-10**: `main.py` combined with `pipeline.py` forms an end-to-end runner from raw data and model input to evaluation output, without manual intermediate steps.
- **HL-14**: Runtime and resource measurement now wrap conformance checking, feature extraction, model training, and evaluation; the totals are reported at the end of every run.
- **LL-14.1**: `evaluation.measure_runtime()` uses `time.perf_counter()` around alignment computation, feature extraction, model training, and evaluation.
- **LL-14.2**: `evaluation.measure_memory()` polls `psutil.Process().memory_info()` to track peak RSS during the entire pipeline run.
- **LL-14.3**: Sub-second precision wall-clock measurements are recorded per step (conformance checking, feature generation, training, evaluation).
- **LL-14.4**: Runtime and resource metrics are printed to the console and can be persisted to a structured JSON log file at the end of the run.

## 4.6 Project Infrastructure & Testing

### Covered

- **HL-18**: PM4Py, scikit-learn, pandas, NumPy, and Matplotlib are used as the primary library stack throughout all modules.
- **HL-19 & LL-19.1-19.2**: The project is organized into `io`, `feature_extraction`, `model_training`, `evaluation`, `pipeline`, `visualization`, and a centralized `config` module; Git is used for version control. Note: the conformance logic is consolidated into `feature_extraction` rather than placed in a dedicated `conformance` module. A formal branching strategy per module was not adopted, as we are working with one branch per team member.

- **HL-20**: `requirements.txt` and `README.md` are present in the repository.
- **LL-20.1**: `requirements.txt` as a plain text file is implemented and considered satisfactory for dependency management.
- **HL-11**: An automated pytest test suite covering IO, feature extraction, model training, and evaluation is executable via a single command (`pytest tests/`) and runs in the GitLab CI pipeline on every push or merge.
- **LL-11.3**: `test_generate_prefixes_lengths_and_content` asserts that a prefix of length $k$ contains exactly the first $k$ events of the original trace; the related `test_pooled_extraction_broadcasts_labels_without_leakage` test additionally confirms that outcome activities are gated out of the feature vector.

**Planned in Sprint 3**

- **LL-11.1**: Dedicated pytest fixtures providing a synthetic, fully conforming XES log and a matching Petri net (currently the tests use ad-hoc DataFrame helpers).
- **LL-11.2**: Explicit test assertion that the alignment cost equals 0.0 on a fully conforming synthetic dataset.
- **HL-21**: Technical documentation of system architecture.

# 5 User Guide

In this sprint, we did not update the existing User Guide, as we focused heavily on the implementation and considered the CLI sufficient for testing from a reviewer's perspective. We plan to write a comprehensive User Guide in Sprint 3, so that a user with no prior process mining experience can operate, understand, and interpret the tool's functionalities.

# 6 Team settings and Workflow

## 6.1 Changes in Responsibilities

As already stated in the first sprint review, we rotated responsibilities for this sprint. Cem was responsible for IO, Model Training, and Feature Extraction,

while Richard was responsible for the new visualization module, evaluation, and pipeline. Both of us were jointly responsible for the CLI and config modules, as they are relatively small and we wanted to work on them together to maintain a shared understanding of the user interface and configuration options. This division of responsibilities worked well, as it allowed each of us to understand and reflect on the other's past contributions and to extend them with new capabilities.

For the next sprint, we plan to keep the same split on the implementation side and place a stronger focus on documentation. We intend to work together on the User Guide, the technical documentation, the README, and the phase review. This will allow us to review each other's content under the four-eyes principle, ensuring more robust output quality.

### 6.2 Workflow Improvements for the next sprint

In this sprint, we revised our branching and workflow strategy. First, we adopted a more granular decomposition of the work items in Git, which allowed us to precisely track and address the planned requirements for this sprint. Second, we consolidated our personal branches into three key branches – `development`, `documents`, and `main` – and created dedicated branches per work item, which were later merged into their respective parent branches. We will continue with this approach, as it has worked very well so far.

For the next sprint, we plan to adopt an even more issue-based workflow on the code side and a more granular approach to both the creation and the content of the documents. As this is the final sprint, all documents are also intended to be completed within the deadline.

## 7 Exemplary Visualizations and Outputs

### 7.1 Visualizations

Even though we did not include visualizations in our requriements when planning this project, it has become evident, that visualizations are a key part of our project. In process and business intelligence, visualizations are of key importance to highlight and contextualize raw numbers and make them accessible to human pattern recognition. Thus we devoted a great amount of time this sprint to create new visualizations and we plan on implementing even more to achieve a great basis for the upcoming evaluation and assessment. **Covered**

- **Process Model Visualization**: `visualize_process_model` supports interactive viewing or saving of normative models (either discovered or provided) in three formats: Petri Net, BPMN, and Process Tree.
- **Decision Tree Visualization**: `visualize_decision_tree` renders the fitted scikit-learn Decision Tree (classifier or regressor), clearly showing the splitting criteria, node purity, and leaf values. This is crucial for the "white-box" interpretability requirement. An example output can be found in `figures/decision_tree.png`.
- **Confusion Matrix Heatmap**: `plot_confusion_matrix_image` generates a clear, color-coded heatmap (blues colormap) of the confusion matrix for classification tasks. The matrix contrasts predicted vs. actual outcomes, highlighting true positives and false positives. An example output can be found in `figures/confusion_matrix.png`.
- **Per-Class Accuracy Chart**: `plot_per_class_bar_chart` complements the confusion matrix by rendering a stacked bar chart displaying the percentage of correct (green) versus incorrect (red) predictions for each class label. An example output can be found in `figures/per_class_accuracy.png`.
- **Processing Times Distribution**: `plot_processing_times` creates a pie chart that visualizes the proportion of time spent in each pipeline phase (e.g., IO, Alignment, Model Training). An example output can be found in `figures/runtime_dist.png`.
- **Performance by Prefix Length**: `plot_evaluate_by_prefix_length` plots prediction metrics (Accuracy, F1 Score, or MAE for regression) on the Y-axis against the prefix length on the X-axis. This visually demonstrates whether adding more events to a trace improves the model's predictive power. An example output can be found in `figures/by_len_vis.png`.

**Planned in Sprint 3**

- **Feature Importance Comparison**: `plot_feature_importance` will render a bar chart of the top predictive features, color-coding conformance-derived features to visually prove their impact versus standard control-flow features.
- **ROC and PR Curves**: `plot_roc_curves` and `plot_pr_curves` will be implemented to overlay the curves of baseline vs. conformance-aware models, providing a standard ML evaluation visualization for binary risk targets.

- **Resource Scaling Plot**: `plot_resource_usage` will plot runtime and memory usage as a function of log size (number of cases/events) to empirically demonstrate the computational overhead of alignment extraction.

## 7.2 CLI Output

To better understand how these visualizations integrate into the user flow, here is an exemplary execution of the interactive CLI that generates the performance evaluation:

```
=== Conformance Predictions ===
Do you have a normative process model? (y/N):
Enter the path to the event log (XES/CSV): data/sepsis-cases.xes
Model discovery will be used.

Which discovery algorithm should be used?
1. Inductive Miner (recommended)
2. Alpha Miner
3. Genetic Miner
Select option (1-3): 2
Alpha Miner has no configurable parameters.

Do you want a visualization of the discovered normative model saved (s) or direc
--- Loading event log: data/sepsis-cases.xes ---
--- Discovering model using ALPHA ---

What should be predicted?
1. Future conformance deviations
2. Process outcome
3. Remaining time
Select option (1-3): 2

Define the target outcome:
Enter activity names that signify a outcome (one per line).
Type 'done' when finished.
> Release A
> Release B
> Release C
> Release D
> Release E
```

```
> done

What should the prediction be based upon?
1. Plain trace (control-flow)
2. Trace and alignment-based diagnostics
3. Case features
4. Mixture of all available information
Select option (1-4): 4

[... Case attributes omitted for brevity ...]

Prefix mode?
  [Enter] none      (default - one row per case, cheapest; no per-prefix alignme
  1)      exactly k (one prefix per case at length k)
  2)      varying 1..k (pooled - prefixes of length 1..k)
Select (Enter/1/2): 2
  max length k (blank = median case length): 8
  step (default 1): 2
  using varying prefixes 1..8, step=2

Enter a split value for the train- / test-split (default=0.7):
Invalid choice. Defaulting to 0.7
--- Extracting target (per case): PROCESS_OUTCOME ---
--- Generating prefixes (min=1, max=8, step=2, gate=on) ---
--- Extracting features over 2772 prefixes: HYBRID ---
--- Preprocessing and encoding data ---

Which ML algorithm do you want to apply on your data?
(1) Decision Tree
(2) Algorithm 2
(3) Algorithm 3
Select option (1-3): 1
Decision Tree selected.

Do you want to configure the parameters yourself? (y/N):

Training model...
Do you want the decision tree saved (s), displayed (d) or not shown (N)? (s/d/N)
Do you want to reconfigure the hyperparameters and retrain? (y/N):

Starting evaluation pipeline...
```

```
--- Using pre-trained model ---
--- Evaluating model ---

Pipeline completed successfully!

Classification metrics (Global Pooled - Lengths: [1, 3, 5, 7]):
  accuracy: 0.6894060995184591
  precision: 0.5921423288201567
  recall: 0.6894060995184591
  f1: 0.6344279306480443
  roc_auc: 0.5942135054989948
  brier_score: 0.0

Overall Confusion Matrix (Pooled over lengths [1, 3, 5, 7]):
                         Release A          Release B          Release C
Release A                   771                4                  0
Release B                    57                0                  0
Release C                    23                0                  0
Release D                    33                0                  0
Release E                    12                0                  0
uncertain_outcome           150                0                  0

Per-Class Prediction Breakdown:
  Release A: 90.1% correct (771/856),  9.9% incorrect (85/856)
  Release B: 0.0% correct (0/76),  100.0% incorrect (76/76)
  Release C: 0.0% correct (0/28),  100.0% incorrect (28/28)
  Release D: 0.0% correct (0/36),  100.0% incorrect (36/36)
  Release E: 0.0% correct (0/12),  100.0% incorrect (12/12)
  uncertain_outcome: 37.0% correct (88/238),  63.0% incorrect (150/238)

Metrics by prefix length:
 prefix_length   n  accuracy  precision   recall       f1  roc_auc  brier_score
             1 326  0.656442   0.579763 0.656442 0.615696 0.584056          0.0
             3 326  0.690184   0.601240 0.690184 0.640255 0.597140          0.0
             5 313  0.683706   0.585921 0.683706 0.627627 0.600362          0.0
             7 281  0.733096   0.575795 0.733096 0.638572 0.581269          0.0

Enter a prefix length to see its specific confusion matrix (or 'N' to skip): n

Do you want to save a visualization of the performance per prefix-length? (y/N):
```

# 8 Sprint review

## 8.1 Richard Schuppener

I perceived this sprint as a great step forward to a finished product. Since we changed responsibilities on who contributes to which module, we both got to look at the other parts of the project where we did not write any code in the first sprint. This deepened our understanding of the codebase significantly and I believe we gained a better overall perspective. I personally was responsible for refactoring what we produced in Sprint 1 such that a *Visualization* module was extracted. This was rather tedious but definitely improved the codebase structure. I also wrote many new visualizations and got to work on the *Evaluation* module. Especially the testing pipeline evolved immensely, since we now have - thanks to Alessandro - a GitLab runner that is available 24/7. We also abolished our previous development branching strategy and moved to the superior methodology of creating descriptive and feature-/ requirement-targetted work items and then creating feature branches that resolve the issue or task we specified. Our progress is neatly tracked in the milestones tab of GitLab. In this manner, we could really utilize the strength of GitLabs rich ecosystem of work items, requirements, milestones, feature-branches, CI etc. The collaboration with Cem was again flawless and even though we were both restricted time-wise (two of my family members sadly passed away), we managed to pull through and deliver properly on our set-out plans from Sprint 1. I want to note that Cem contributed a very significant amount to the Sprint 2 review document, before I could even start working on it. He also went out of his way to organize new work items at the beginning of Sprint 1, so that we could easily track our progress on the tasks we planned in the previous sprint review. We plan to finish all remaining requirements and features in Sprint 3. I personally want to clean up the codebase, making it as simple as possible and properly modular. I believe we still have enough work to do in polishing our product, but we also made great progress.

## 8.2 Cem Colak

In this sprint we switched the responsibilities. This was a very important step for me, to really understand the other modules of our project and add/improve improve some functionalities. I feel very comfortable with our codebase, and even see some potential implementation improvements we can make in the next sprint. Furthermore, the new branching and granularity was pretty new for me, although that I know, that it is common pratice. I

really liked the approach and will definitely use it in my future projects as well. Additionally, I noticed that preprocessing is a pretty heavy module and understand now, why Prof. Van der Aalst meant, that Preprocessing makes a huge stake in the whole data pipeline. In terms of time-management, I was a bit restricted in the second week, since I am on vacation. Still, Richard and me already planned and timed our engagement in the whole sprint such that we were able to manage all our goals for the second sprint in a smooth and timely manner. The only thing I didn't expect is the weak level of documentation we have had so far. On the other hand, I was really busy in the past days with my seminar submission and all the assignments, so I think it is a valid reason why we achieved less in this part of the project. For the next sprint, we plan to focus on the documentation from the beginning, as we have alrady implemented most of the functionalities and can produce a valuable documentation base. In terms of the collaboration with Richard I'm very satisfied overall and valued his engagement in planning and adivsing a lot. We have had regular conversations and calls such that our boths views are aligned and no one works apart from the other. I'm looking foward continuing, polishing, and finishing the software artifact with Richard in the incoming last sprint.